Embeddings y similitud semántica

¿Distingue un modelo de embeddings el significado de la coincidencia de palabras clave? · Versión 2

Curso: AI Engineering · Coderhouse **Módulo 3:** Persistencia de datos y vector DBs **Autor:** Javier Acuña

> **Qué cambió respecto de la v1.** Se amplió el conjunto de 7 a 12 oraciones etiquetadas (relevantes, irrelevantes y contradictorias), se sumó una consulta negada, un re-ranker que lee consulta y documento juntos, la comparación de tres variantes de embeddings, métricas (precisión@k, margen y umbral calibrado), las formas de las matrices con salidas reales en el código y un diagrama que rotula la búsqueda k-NN dentro de la base vectorial. Las 7 oraciones originales se conservan: T1 pasa a llamarse I1 y T2 pasa a llamarse C1.

1 Objetivo y metodología

El objetivo es comprobar si una búsqueda recupera los textos que **significan** lo que pide una consulta o si, como un buscador tradicional, se deja llevar por las **palabras** que comparten. En esta versión se agrega una tercera pregunta: ¿qué pasa cuando el documento habla del mismo tema pero **dice lo contrario**?

Se compararon tres métodos sobre el mismo conjunto y las mismas dos consultas: (1) **palabras clave**, con vectores TF-IDF y similitud coseno; (2) **embeddings** de Google Gemini, con similitud coseno, en tres variantes (`gemini-embedding-001` con 3072 y 768 dimensiones, y `gemini-embedding-2-preview`); y (3) un **re-ranker**: un LLM (`gemini-3.8-flash`, temperatura 0) que recibe la consulta junto con cada documento y devuelve, con salida estructurada en Pydantic, un puntaje de relevancia de 0 a 1 y un motivo. Para cada método y consulta se calcularon la precisión@3, la precisión@5, el margen de separación y el umbral que mejor separa relevantes de no relevantes.

2 Conjunto de evaluación

Doce oraciones en tres grupos. **R** describe el despliegue de microservicios sin repetir el vocabulario de la consulta. **I** es irrelevante, desde la trampa evidente de palabras clave (I1) hasta trampas más sutiles: software pero de otro tema (I2, I3) o vocabulario de «publicar en plataformas» en otro dominio (I4). **C** habla de lo mismo, pero lo niega o lo abandona.

Id	Oración	Rol en el conjunto
R1	Publicamos cada componente del backend como un contenedor independiente en el clúster de Kubernetes.	Despliegue con vocabulario de infraestructura (contenedor, clúster, Kubernetes).
R2	La aplicación está dividida en servicios pequeños que se ponen en producción por separado.	Definición en lenguaje llano: servicios pequeños que van a producción por separado.
R3	Cada módulo de la plataforma se empaqueta con Docker y se libera sin afectar al resto del sistema.	Foco en el empaquetado (Docker) y la liberación independiente de cada módulo.
R4	Nuestro pipeline de CI/CD lleva automáticamente cada API autónoma desde el repositorio hasta los servidores.	Foco en la automatización: un pipeline de CI/CD que lleva cada API a los servidores.
R5	En lugar de un monolito, entregamos piezas de software desacopladas que escalan cada una por su cuenta.	Contraste arquitectónico: piezas desacopladas en lugar de un monolito.
I1	Contratamos un servicio de micro limpieza con despliegue de personal en todo el edificio.	Trampa de palabras (ex T1): comparte «servicio», «micro» y «despliegue», pero trata de limpieza.

Id	Oración	Rol en el conjunto
I2	Refactorizamos las funciones del backend para que el código sea más fácil de leer.	Trampa sutil: es software y backend, pero de calidad de código, no de despliegue.
I3	El equipo de diseño actualizó la paleta de colores de la aplicación móvil.	Trampa sutil: es una aplicación, pero de diseño visual.
I4	Los servicios de streaming publican sus series nuevas cada viernes en distintas plataformas.	Trampa sutil: «servicios», «publican», «plataformas», en el dominio del streaming.
C1	No hacemos despliegue de microservicios: todo el sistema es un único monolito.	Contradicción explícita (ex T2): contiene la consulta textual, pero la niega.
C2	Volvimos a un monolito porque mantener servicios separados en producción era demasiado costoso.	Contradicción por abandono: volvieron al monolito. No usa la palabra «microservicios».
C3	Desistimos de partir la aplicación: seguimos publicando un único ejecutable con todo el sistema.	Contradicción por decisión: no partir la aplicación, un único ejecutable.

¿Por qué el embedding separa a I1 del grupo relevante?

En la v1, I1 (ex T1) quedó última aunque comparte tres palabras con la consulta. La matriz de similitud entre documentos lo explica: I1 se parece a las relevantes en promedio **0,729**, mientras que las relevantes se parecen entre sí **0,833**. Hay tres rasgos del texto que la alejan:

1. El dominio. Las palabras que definen el tema de I1 son *limpieza*, *personal* y *edificio* (servicios generales), no software. **2. La ausencia de entidades técnicas.** Todas las R comparten un campo semántico (backend, contenedor, Docker, Kubernetes, API, pipeline, producción, software) del que I1 no tiene ni un término. **3. La polisemia resuelta por contexto.** Un modelo contextual no asigna un vector fijo a «despliegue» o «servicio»: «despliegue *de personal*» y «servicio *de micro limpieza*» se interpretan por sus vecinas, igual que el ejemplo de «banco» de la unidad. Por eso la coincidencia literal aporta poco.

Las trampas nuevas lo confirman con una gradación. I2 (refactor del backend) es la irrelevante más cercana al grupo R (0,784), porque comparte el dominio de software. Aun así, frente a la consulta queda en 0,655, debajo de todas las R: el modelo distingue «mejorar el código» de «desplegar servicios». I4 (streaming), con vocabulario de publicación, queda en 0,638.

3 Experimento 1 · consulta afirmativa

Consulta: **“despliegue de microservicios”**. Respuestas correctas: R1–R5.

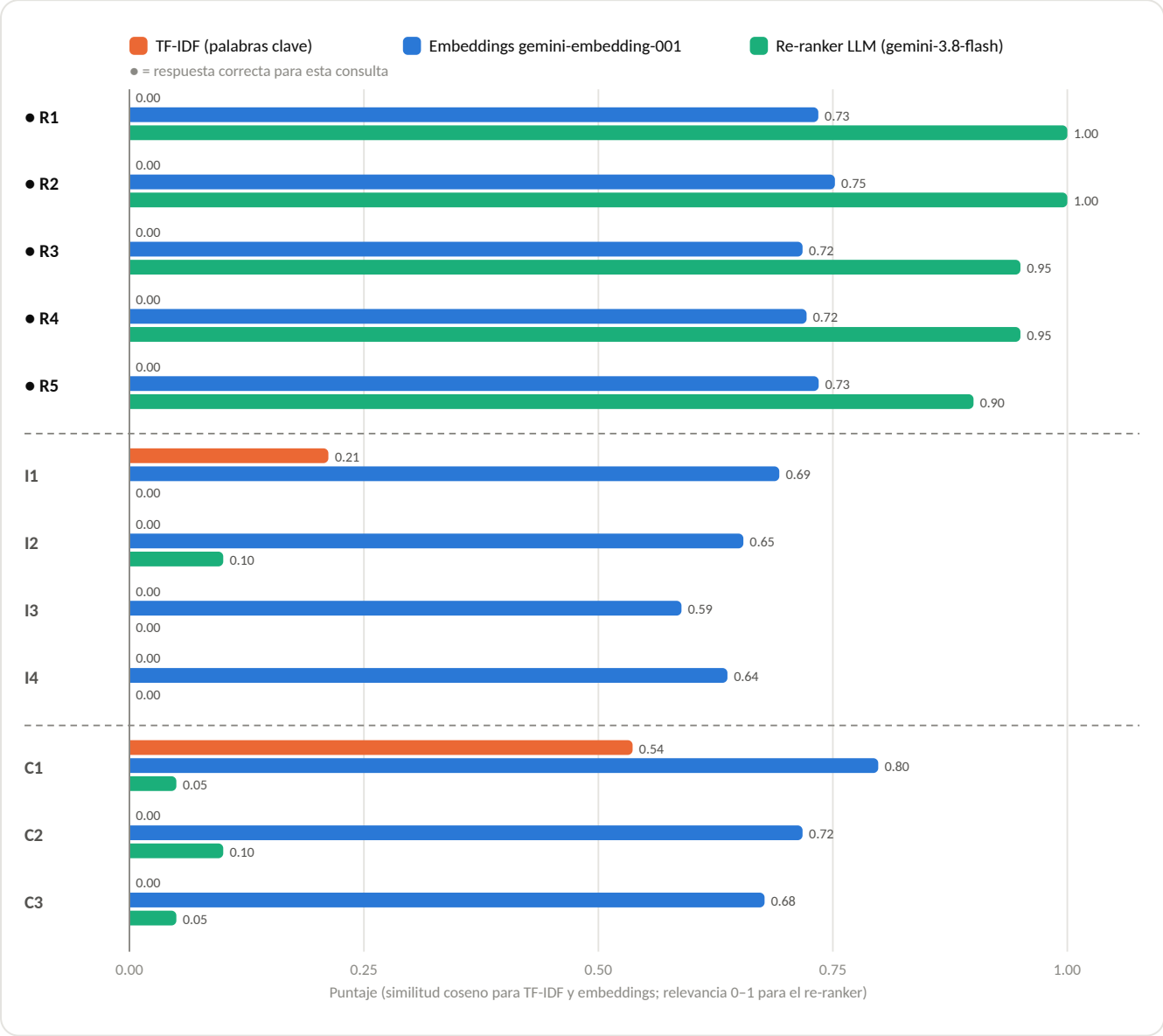


Figura 1. Puntaje de cada documento según los tres métodos. Las líneas punteadas separan los grupos R, I y C.

TF-IDF solo «encuentra» dos documentos: C1 (0,54), que contiene la consulta literal pero la niega, e I1 (0,21), la trampa de palabras. Las cinco R obtienen 0. Precisión@3 y @5 = 0: el buscador por palabras clave no devuelve ni una respuesta correcta.

Embeddings (001) recuperan las cinco R (entre 0,718 y 0,752) y dejan a las cuatro I al fondo. Pero C1 vuelve a quedar **primera (0,798)**, y C2 (0,718) se mete entre las relevantes, empatada con R3. El dato más revelador es el promedio por grupo: relevantes **0,732** y contradictorias **0,731**. Para este modelo, «hacemos microservicios» y «abandonamos los microservicios» son prácticamente lo mismo. El tema le gana a la postura.

El re-ranker resuelve el caso: R1–R5 entre 0,90 y 1,00, y todo lo demás en 0,10 o menos. Las C caen a 0,05–0,10 con motivos como «Niega expresamente el uso y despliegue de microservicios» (C1). Al leer consulta y documento juntos, evalúa si el documento *responde*, no solo si se parece.

4 Experimento 2 · consulta negada

Consulta: **“aplicación monolítica sin microservicios”**. Ahora las respuestas correctas son C1–C3, y R1–R5 pasan a ser las que contradicen la consulta.

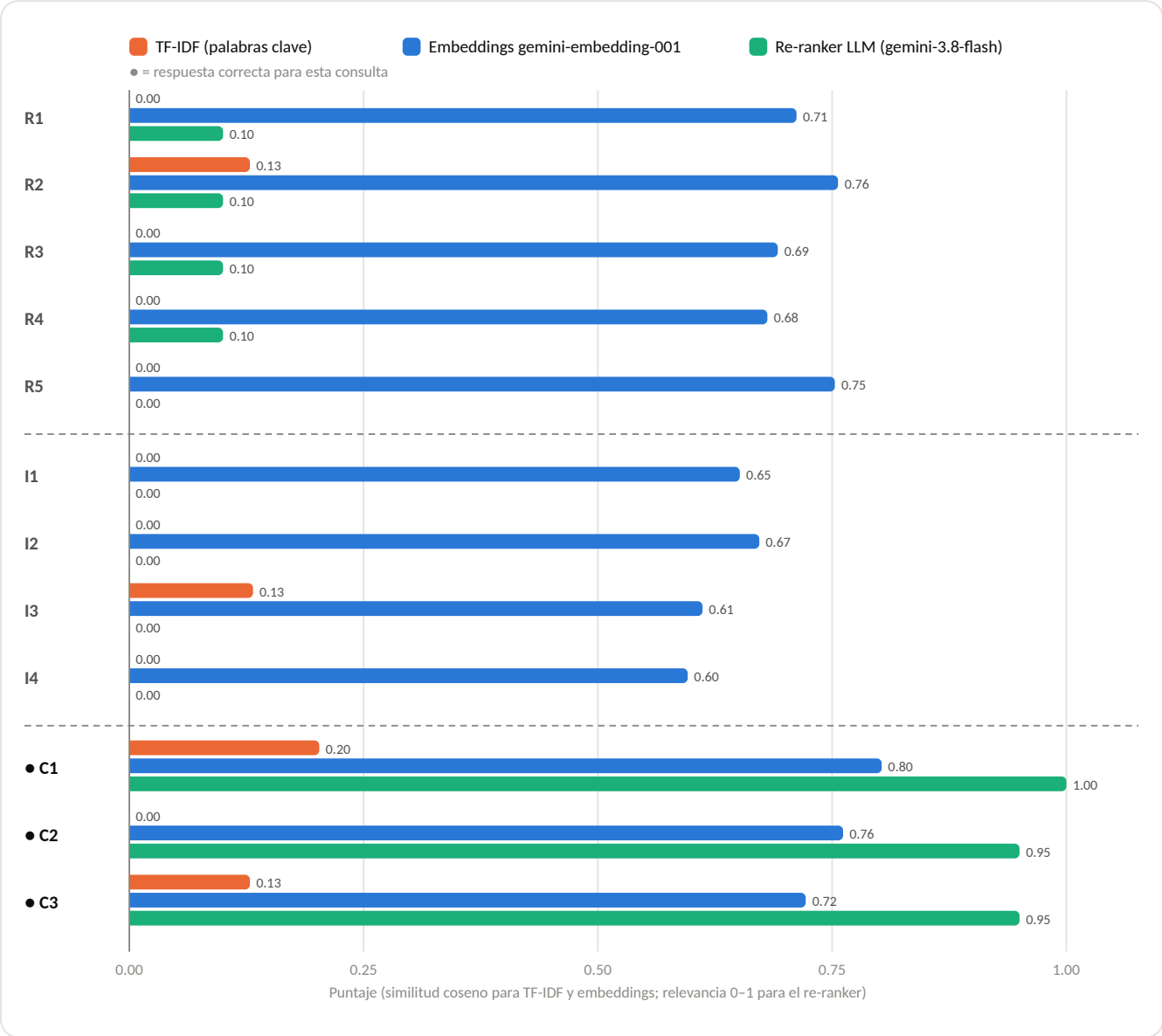


Figura 2. Mismos documentos y métodos, con la consulta negada.

TF-IDF muestra otro límite: no reconoce variantes de una misma palabra. La consulta dice «monolítica» y los documentos dicen «monolito», así que C2, una respuesta correcta, obtiene 0. En cambio, I3 (diseño de una app móvil) aparece segunda solo por compartir «aplicación».

Embeddings (001) se mueven en la dirección correcta: C1 (0,803) y C2 (0,762) suben al tope y el grupo C supera al R en promedio (0,762 contra 0,719). Pero R2 (0,756) y R5 (0,753), que **contradicen** la consulta, quedan por encima de la correcta C3 (0,722). La negación «sin microservicios» se diluye: la palabra «microservicios» empuja hacia los documentos que sí los usan.

El re-ranker invierte por completo su criterio y acierta: C1–C3 entre 0,95 y 1,00, R1–R5 entre 0 y 0,10. Sobre R5 explica que «descarta explícitamente el uso de un monolito a favor de piezas desacopladas». Es el mismo texto que en el experimento 1 puntuó 0,90: el puntaje depende de la consulta, como debe ser.

5 Calibración de umbrales y precisión@k

Para pasar de «parece que funciona» a una decisión medible, cada método se evaluó con las etiquetas del conjunto. La **precisión@k** indica qué proporción de los k primeros resultados es correcta. El **margen** indica si existe algún umbral que separe perfectamente lo relevante de lo demás. El **umbral calibrado** es el corte que, en este conjunto, clasifica mejor.

	Consulta afirmativa (5 relevantes)					Consulta negada (3 relevantes)				
Método	**P@3**	**P@5**	**Margen**	**Umbral**	**Exact.**	**P@3**	**P@5***	**Margen**	**Umbral**	**Exact.**
TF-IDF	0.00	0.00	-0.54	0.536	50%	0.33	0.40	-0.13	0.203	83%
Embeddings 001 · 3072	0.67	0.80	-0.08	0.722	83%	0.67	0.60	-0.03	0.762	92%
Embeddings 001 · 768	0.67	0.80	-0.08	0.703	83%	0.67	0.60	-0.03	0.748	92%
Embeddings 2-preview · 3072	0.67	0.80	-0.03	0.729	92%	0.67	0.40	-0.08	0.779	92%
Re-ranker LLM	1.00	1.00	**+0.80**	0.900	100%	1.00	0.60	**+0.85**	0.950	100%

Margen = puntaje del peor documento relevante − puntaje del mejor no relevante (positivo: existe un umbral que separa perfecto). **Umbral** = corte que maximiza la exactitud en este conjunto. *Con 3 relevantes, la P@5 máxima posible en la consulta negada es 0,60.

Ningún método de similitud pura logra margen positivo: siempre hay al menos un documento no relevante que puntúa más que uno relevante. Por eso, con embeddings, el mejor umbral posible igual se equivoca (83–92 % de exactitud), y sus errores son siempre las contradicciones. Hay que leer la exactitud con cuidado: en la consulta negada hay solo 3 relevantes de 12, así que marcar todo como «no relevante» ya da 75 %. El 83 % de TF-IDF ahí es casi trivial.

El re-ranker es el único método con margen positivo en las dos consultas (+0,80 y +0,85) y 100 % de exactitud con umbrales altos (0,90 y 0,95). En la práctica significa que su umbral es estable y fácil de fijar, mientras que el de los embeddings varía con cada modelo (0,703, 0,722 y 0,729 en la consulta afirmativa) y deja errores igual.

6 Comparación de modelos de embeddings

Se compararon tres variantes para ver cómo cambian los márgenes al cambiar el modelo. Recordatorio de la unidad: los puntajes de modelos distintos **no se comparan entre sí en valor absoluto**, porque cada uno vive en su propio espacio vectorial. Lo que sí se compara son las **diferencias entre grupos** dentro de cada modelo.

Modelo	Dim.	Prom. R	Prom. C	Prom. I	R − C	R − I	Umbral
Embeddings 001 · 3072	3072	0.732	0.731	0.643	+0.001	+0.089	0.722
Embeddings 001 · 768	768	0.715	0.712	0.616	+0.002	+0.099	0.703
Embeddings 2-preview · 3072	3072	0.755	0.701	0.570	+0.054	+0.185	0.729

Promedios de similitud con la consulta afirmativa por grupo. **R − C** mide cuánto distingue el modelo «a favor» de «en contra»; **R − I**, cuánto distingue el tema correcto de un tema ajeno.

3072 contra 768 dimensiones (mismo modelo). Los primeros valores del vector son idénticos en ambas versiones: el modelo de 768 es un recorte del de 3072. Los puntajes bajan unos 0,02 de forma pareja y el ranking prácticamente no cambia: márgenes de −0,083 contra −0,081 y de −0,035 en ambos casos. Usar 768 dimensiones ocupa **4 veces menos espacio** en la base vectorial sin pérdida visible en este conjunto.

gemini-embedding-2-preview separa mejor por tema: la diferencia R − I sube de 0,089 a 0,185, y en la afirmativa pone a R2 y R1 por encima de C1 y logra el mejor margen de los embeddings (−0,032). También distingue algo más la postura (R − C de +0,054 contra +0,001). Pero en la consulta negada rinde peor que el 001 (margen −0,081, P@5 0,40), porque R5 y R2 vuelven a colarse entre las correctas. Ningún modelo de embeddings resuelve la negación: la mejora entre modelos es de grado, no de naturaleza.

Con 12 documentos y 2 consultas, estas diferencias son indicativas y no concluyentes. Para elegir modelo en un proyecto real habría que repetir la medición con un conjunto de evaluación más grande y del dominio propio.

7 Cálculo de la similitud coseno con scikit-learn

Cuatro fragmentos mínimos para reproducir el cálculo. La consulta es una matriz de **1 × dimensión** y los documentos una de **n × dimensión**. `cosine_similarity` compara cada fila de la primera contra cada fila de la segunda y devuelve una matriz de **1 × n**. Se asume `emb = GoogleGenerativeAIEmbeddings(model="gemini-embedding-001")`.

1 · Vectores y formas de las matrices

```
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

vec_query = np.array(emb.embed_query("despliegue de microservicios")).reshape(1, -1)
vec_docs  = np.array(emb.embed_documents(textos))      # textos: lista de 12 oraciones

print(vec_query.shape, vec_docs.shape)
print(vec_query[0][:5].round(4))
```

SALIDA

```
(1, 3072) (12, 3072)
[-0.0089  -0.0155  0.0145  -0.0670  0.0140]
```

2 · Similitud coseno con scikit-learn

```
sims = cosine_similarity(vec_query, vec_docs)    # (1 x 3072) contra (12 x 3072) -> (1 x 12)
print(sims.shape)
print(sims[0][:3].round(4))                       # R1, R2, R3
```

SALIDA

```
(1, 12)
[0.7345  0.7522  0.7177]
```

3 · Top-k: los documentos más parecidos

```
ids  = ["R1", "R2", "R3", "R4", "R5", "I1", "I2", "I3", "I4", "C1", "C2", "C3"]
top3 = np.argsort(sims[0])[::-1][:3]              # índices de mayor a menor
print([ids[i] for i in top3])
```

SALIDA

```
['C1', 'R2', 'R5']
```

4 · La misma fórmula a mano (verificación)

```
def similitud_coseno(a, b):
    """cos(θ) = (A · B) / (‖A‖ · ‖B‖)"""
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

print(round(similitud_coseno(vec_query[0], vec_docs[0]), 4))   # igual a sims[0][0]
```

SALIDA

```
0.7345
```

Las salidas son las reales de la corrida con `gemini-embedding-001`. El script completo, que además calcula TF-IDF, re-ranker y métricas, es `similitud_semantica.py`.

8 Diagrama de flujo: búsqueda semántica con re-ranking

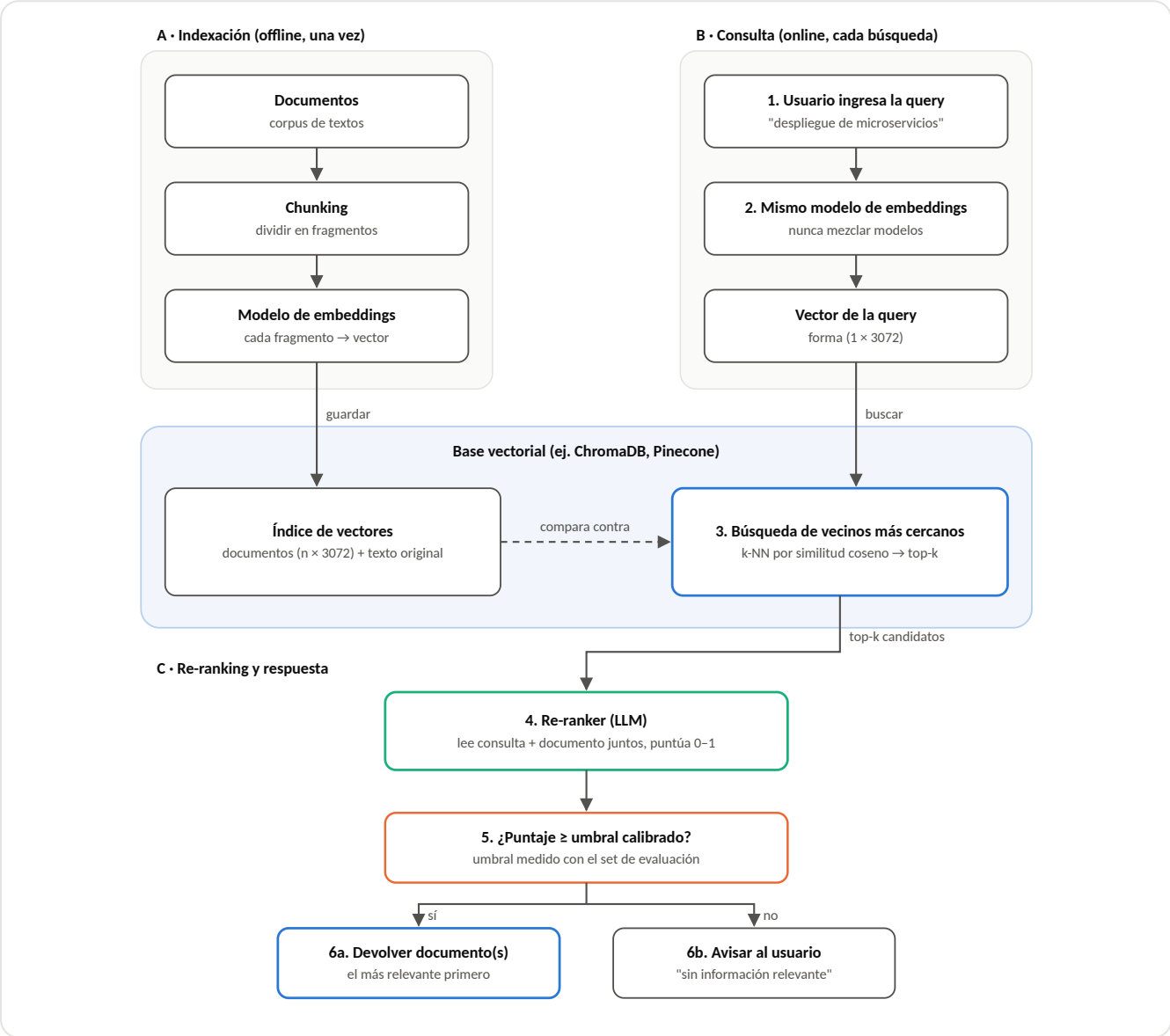


Figura 3. A: los documentos se vectorizan una sola vez y se guardan en el índice. B: cada query se vectoriza con el mismo modelo. Dentro de la base vectorial, la búsqueda de vecinos más cercanos (k-NN por similitud coseno) devuelve el top-k, que es barato y con buen recall. C: el re-ranker relee esos k candidatos junto con la query y corrige errores como la negación. Solo pasa lo que supera el umbral calibrado.

9 Conclusiones

- **Palabras clave:** no sirven como búsqueda principal. No recuperan textos escritos con otro vocabulario (P@5 = 0 en la consulta afirmativa), no reconocen variantes de palabra («monolito» contra «monolítica») y premian coincidencias superficiales.
- **Embeddings:** resuelven la rigidez terminológica y descartan bien lo irrelevante, incluso las trampas sutiles del mismo dominio (I2). Pero agrupan por **tema**, no por **postura**: con el modelo 001, las relevantes y las contradictorias obtienen el mismo puntaje promedio (0,732 contra 0,731).
- **Re-ranker:** leer consulta y documento juntos resuelve la negación en las dos consultas (margen +0,80 y +0,85, P@3 = 1,00). Su costo es una llamada al LLM, por eso se aplica solo sobre el top-k que devuelve la búsqueda vectorial.
- **Modelos de embeddings:** cambiar de modelo mueve los márgenes y los umbrales (0,703 a 0,729), así que el umbral nunca se copia de un modelo a otro: se calibra con un conjunto de evaluación. Recortar a 768 dimensiones mantuvo la calidad con 4 veces menos almacenamiento.
- **Arquitectura recomendada:** búsqueda k-NN con embeddings para el *recall*, re-ranker para la *precisión* y umbral calibrado con un conjunto etiquetado de casos relevantes, irrelevantes y contradictorios, que habría que ampliar con datos reales antes de producción.